PER SCHOLAS

CAP 931: Capstone - Build a Sales Agent Prototype Using Multi-Agent GPT Models

Version 1, 9/19/2024

Open in new window

Introduction:

Create a prototype of a sales assistant agent that helps a new sales representative gain insights into a prospective account (client), understand which competitors the account might be working with, and grasp the company's strategy in the area the representative is trying to sell. This task will assess your coding skills, understanding of the LLM space, ability to work with models like GPT, and your ability to think, experiment, and be resourceful.

Objectives:

1. Develop a Sales Assistant Agent: Create a functional prototype that can assist a sales representative by generating account insights such as company strategy, competitor mentions, and leadership information.
2. Leverage LLMs (GPT Models): Select and implement an appropriate GPT model (e.g., GPT-3.5 or GPT-4) to process input data and generate insights.
3. Streamlit Interface: Build a user-friendly Streamlit interface for input collection, enabling sales reps to enter key details such as product name, company URL, and competitors.
4. Data Processing & Integration: Ensure the system can parse through provided URLs, extract relevant web data, and refine the LLM outputs for relevance and accuracy.
5. Output Generation: Design the output as a one-page document summarizing account insights, incorporating company strategies, competitor analysis, and leadership information.
6. Experimentation & Enhancement: Apply prompt engineering techniques and experiment with different prompts to improve the quality and usefulness of the outputs.
7. Optional Features: Propose or implement optional features such as an alert system or production deployment.

Requirements:

Before building the sales assistant agent, it is essential to establish a solid technical foundation. The instructions section will guide you through the steps required to configure the development environment, ensuring that it is equipped with the necessary tools and access to the selected large language model (LLM). Setting up these components correctly will enable smooth interactions between the sales rep's inputs and the LLM, streamlining the process of generating insights. The following steps will walk you through the process of accessing the LLM, configuring the programming environment, and setting up the user interface with Streamlit.

- Access to an LLM (GPT3.x, 4.x), please refer to this guide for setting up OpenAI trial account.
- Programming Language: Python (preferred for compatibility with LLMs like GPT).
- You can use any publicly available IDE with Python support, such as:
  - Replit: https://replit.com
  - Google Colab: https://colab.research.google.com
  - Jupyter Notebook: https://jupyter.org
- Preferred Stack: Use Streamlit to create a simple, user-friendly interface. Streamlit is free, you can also find the official documentation for setting up Streamlit here: Streamlit Installation Guide. The installation guide will help you create your first app and get familiar with the basic setup.

1. Instructions:

Install Streamlit using this command in your chosen Python IDE terminal:

```
pip install streamlit
```

Note: Our sales assistant agent will only respond to the specific use case scenarios defined within this project such as generating insights about a prospective account, competitors, and company strategies. It will not engage in or respond to general conversations or unrelated inquiries. This is achieved by applying prompt constraints and context-setting techniques to ensure that the GPT model only processes inputs relevant to the task at hand. By carefully crafting prompts and using clear delimiters, we prevent the agent from deviating into non-use case responses.

2. Inputs:

Set up an input section where a sales rep can provide the following information. These inputs will provide the necessary data for the system to generate meaningful insights using the LLM. The Streamlit interface will act as the front-end where the sales rep enters these details, and the backend will process them to provide outputs. The following fields are dynamic, which means that the students will have the choice to assume a product of their liking, based on it they can associate other details to it as well.

- Product Name: What product are you selling?
- Company URL: The URL of the company you are targeting. (Use this to derive the company ID and other metadata.)

- Product Category: This could be one word or a sentence (e.g., "Data Warehousing" or "Cloud Data Platform"). The LLM should identify the category from the description.
- Competitors: URLs of competitors (similar to the company URL input).
- Value Proposition: A sentence summarizing the product's value.
- Target Customer: Name of the person you are trying to sell to.
- Optional: Upload a product overview sheet or deck. The system should parse through this document to extract more insights into the product.

3. LLM Capabilities:

- Model Selection: Choose an LLM (e.g., GPT-3.5, GPT-4) to handle the natural language processing (NLP) tasks. Justify your choice based on factors such as accuracy, cost, speed, and any specific strengths/weaknesses of the model.
- Prompt Engineering: Experiment with different prompt structures or chaining techniques to refine the LLM outputs.
- Data Integration: Use the input URLs and context to refine the LLM outputs further, ensuring they are accurate and relevant.

4. Outputs:

The LLM (e.g., GPT-3.5 or GPT-4) will generate a simple one-pager that provides the sales representative with the following insights, based on publicly available web data. This one-pager should be generated by the LLM using the inputs from step 2 provided in the Streamlit interface (e.g., Product Name, Company URL, Competitors, etc.). Use perscholas template to create this one pager document.

  - Company Strategy: A summary of the company's activities in the industry relevant to the product being sold. For example:
    - Mention any public statements, press releases, or articles where key executives (e.g., Chief Data Officer, Chief Compliance Officer) have discussed relevant topics.
    - Summarize job postings or other indicators that hint at the company's strategy or technology stack (e.g., skills required in job postings).
  - Competitor Mentions: Any mention of the competitors provided in the input.
  - Leadership Information: Key leaders at the prospect company and their relevance (e.g., those quoted in press releases over the last year).
  - Product/Strategy Summary: For public companies, include insights from 10-Ks, annual reports, or other relevant documents available online.
  - Article Links: Provide links to full articles, press releases, or other sources mentioned in the output.

5. Optional Enhancements:

- Improving Output Results: Describe or implement an approach to enhance the accuracy, relevance, and usefulness of the outputs.
- Alert System: Design or describe a system that sends alerts (via email) whenever new press releases, job postings, or other relevant content is published that includes user-selected keywords.
- Production Deployment: Outline how you would deploy this app in a production environment, including considerations for scalability, security, and maintenance.
- How would you use different types of models to generate things like a deck for the sales reps meeting, give him more insight if he is able to share some specific documents with you, and so on.

6. Documentation:

- **Technical Documentation:** Please use the Perscholas template to provide clear documentation that explains your code structure, decisions, and how the system works.
- **Time Management:** Document how you allocated your time across different tasks and why.
- **Challenges and Solutions:** Describe any challenges you encountered and how you overcame them.
- **Experiments:** If you experimented with different models, prompts, or approaches, document these experiments and their outcomes.
- **System Outputs:** Include the generated one-pager along with the insights mentioned, and ensure that all sources (e.g., articles, press releases) are well-documented and accessible.

Time/Duration:

This final project is to be completed in 2 days.

Grading:

Criteria	Exemplary (20 pts)	Proficient (10-19 pts)	Needs Improvement (0-5 pts)
Technical Setup	The environment is fully set up with correct LLM access, Python IDE, and a Streamlit interface. Clear, well-organized setup documentation is provided, making the environment easy to replicate.	The environment is mostly set up, but there are minor issues in the LLM access or IDE configuration. The setup is functional, but documentation could be clearer.	The environment is incorrectly or partially set up. Issues with accessing the LLM or using Streamlit. No or insufficient documentation.
Inputs Handling	All required inputs (product name, company URL, product category, etc.) are effectively integrated into the Streamlit interface. Input validation and parsing are done correctly, and the system handles optional uploads seamlessly.	Most required inputs are integrated, but there may be minor validation issues or inconsistencies in handling optional inputs. The interface may lack clarity or ease of use in some areas.	Significant issues with input handling, missing fields, or incorrect validation. The interface is difficult to use or lacks key functionality.
LLM Model Selection & Use	The chosen LLM is well-justified based on accuracy, speed, and cost. Prompts are expertly crafted, with chaining techniques that enhance the output. The model consistently generates accurate, insightful results.	The chosen LLM is adequate, though the justification lacks depth. Prompts are generally effective but may need more refinement. Outputs are relevant but could be more accurate or detailed.	Poor LLM selection, with insufficient reasoning. Prompts are not effectively crafted, resulting in inaccurate or irrelevant outputs.
Data Integration & Output Relevance	The system integrates web data effectively and outputs a highly	Data integration works, but there may be occasional inaccuracies or	The system fails to integrate web data accurately. The one-pager is incomplete,

	relevant, clear one-pager with insights on strategy, competitor mentions, leadership, and product summaries. Data sources are reliable, and outputs are consistently high quality.	missed information in the output. The one-pager is clear but lacks depth in some areas.	with missing or irrelevant insights.
Optional Enhancements (e.g., Alert System, Output Improvement)	Optional enhancements (e.g., alerts, output refinements) are fully implemented with well-thought-out strategies for improvement. These features significantly add value to the project.	Some optional enhancements are present, but they may not be fully implemented or need refinement to be more effective.	Optional enhancements are either missing or poorly implemented, adding little value to the project.
Production Deployment Considerations	A thorough plan for production deployment is provided, addressing scalability, security, and maintenance. The deployment strategy is practical and well-documented.	A deployment plan is included but lacks details in some areas (e.g., scalability, security). The plan is generally feasible but needs further refinement.	No clear plan for production deployment, or the plan provided is unrealistic or lacks key considerations.
Documentation Quality	Documentation is clear, well-organized, and covers all aspects of the project, including technical setup, code structure, time management, challenges, and experiments.	Documentation is generally clear but may be missing some details or depth in areas such as time management or experiment outcomes.	Documentation is incomplete, unclear, or poorly organized. Key aspects of the project are not explained or justified.